Installing the Certificate Enrollment Web Service

michaelwaterman.nl/2026/08/03/part-3-installing-the-certificate-enrollment-web-service-ces

Michael Waterman August 3, 2026

In the previous article, we deployed the Certificate Enrollment Policy Web Service (CEP) and configured support for Kerberos, Username/Password, and Client Certificate authentication. Clients can now successfully retrieve certificate enrollment policies over HTTPS using the XCEP protocol. However, while the policy infrastructure is now in place, clients still have no way to request certificates.

In this article, I'll complete the Microsoft Certificate Enrollment Services architecture by deploying the Certificate Enrollment Web Service (CES). I'll prepare Active Directory, configure a dedicated Group Managed Service Account (gMSA), install IIS, deploy the CES role, configure the supported authentication methods, and validate the deployment using PowerShell. By the end of this article, you'll have a fully operational Certificate Enrollment Web Service capable of securely processing certificate requests over HTTPS using the WSTEP protocol.

In the next article, I'll put everything together by configuring both domain-joined and workgroup clients, registering Certificate Enrollment Policy Servers, and requesting certificates using both the Windows Certificate Enrollment API and PowerShell.

Microsoft Certificate Enrollment Services series

This article is part of a six-part series covering Microsoft Certificate Enrollment Services from architecture and deployment to high availability and troubleshooting.

- Part 1 – Understanding the Architecture
- Part 2 – Installing the Certificate Enrollment Policy Web Service (CEP)
- *Part 3 – Installing the Certificate Enrollment Web Service (CES)*
- Part 4 – Using Certificate Enrollment Services in Practice
- Part 5 – Building a Highly Available CEP/CES Infrastructure
- Part 6 – Troubleshooting, Logging and Event IDs

In this article, I'll build upon the environment created in Part 2. Before continuing, ensure that the Certificate Enrollment Policy Web Service (CEP) is fully operational and that you have a functioning Enterprise Certification Authority (CA) available. For this walkthrough, the following lab environment is used:

Server	Role
lab-dc-01	Active Directory Domain Controller
lab-srv-02	Enterprise Certification Authority
lab-srv-01	IIS Certificate Distribution Point (CDP/AIA)
lab-srv-03	Certificate Enrollment Policy (CEP) Server
lab-srv-04	Certificate Enrollment Web Service (CES) Server
lab-srv-10	Offline Root Certification Authority

Before proceeding, verify that:

- The CES server is joined to the Active Directory domain.
- The Enterprise Certification Authority is online and reachable.
- The IIS application pool identity requires the Log on as a service user right when using a Group Managed Service Account (gMSA). This is the default, but you better check it upfront.
- TCP port 443 (HTTPS) must be accessible to clients, check any intermediate firewalls.

With these prerequisites in place, we can begin preparing Active Directory for the Certificate Enrollment Web Service deployment.

Preparing Active Directory

Most of the Active Directory preparation was completed in Part 2. In this article, I'll extend the existing configuration by granting the future CES server permission to enroll for a TLS certificate, creating a dedicated Group Managed Service Account (gMSA) specifically for CES, and configuring the required Service Principal Names (SPNs) including delegation.

Add the CES server to the security group

Before requesting a TLS certificate, add the future CES server to the existing "*GG-T0-PKI Enroll TLS Certificates*" security group. This grants the server permission to enroll for the TLS certificate template created in the previous article.

```
Add-ADGroupMember `
    -Identity "GG-T0-PKI Enroll TLS Certificates" `
    -Members "lab-srv-04$"
```

After updating the group membership, reboot the server before requesting the certificate to ensure the new permissions are recognized.

Request the TLS certificate

Once the certificate template has been published, request a certificate on the future CES server. The certificate thumbprint will be used later when installing the Certificate Enrollment Policy Web Service.

```
$TemplateName = "LabActiveDirectoryTLSCertificate"

$Certificate = Get-Certificate `
    -Template $TemplateName `
    -CertStoreLocation "Cert:\LocalMachine\My"

$Certificate.Certificate
```

Note! The displayed thumbprint is later used during the installation of the CES service

Configure the Group Managed Service Account (gMSA)

In Part 2, we created the KDS Root Key, enabling the use of Group Managed Service Accounts (gMSAs) within the Active Directory forest. Since the KDS Root Key is already available, I'll now create a dedicated gMSA for the Certificate Enrollment Web Service (CES), following the same approach used for the CEP server.

Create a security group for CES servers

Create a dedicated security group that contains all servers allowed to retrieve the managed password of the CES gMSA. This approach simplifies future expansion by allowing additional CES servers to be added without modifying the gMSA configuration.

```
$GroupName = "GG-T0-CES Servers"

$GroupParameters = @{
    Name            = $GroupName
```

```
    SamAccountName  = $GroupName
    GroupScope      = "Global"
    GroupCategory   = "Security"
    Path            = "OU=Groups,OU=Tier
0,OU=Admin,DC=corp,DC=michaelwaterman,DC=nl"
    Description     = "Members are allowed to retrieve the password for the CES
gMSA."
}

New-ADGroup @GroupParameters
```

Add computer members to the security group

Add the future CEP server to the "*GG-T0-CES Servers*" security group. This grants the server permission to retrieve the managed password of the Group Managed Service Account (gMSA). Run the following PowerShell command:

```
Add-ADGroupMember `
    -Identity "GG-T0-CES Servers" `
    -Members "lab-srv-04$"
```

Note! You must reboot the computer to update the group membership.

Create the group managed service account

With the security group in place, we can create the Group Managed Service Account (gMSA) that will be used by the Certificate Enrollment Web Service (CES). The gMSA eliminates the need for manual password management while providing a dedicated service identity for the IIS application pool. Run the following PowerShell command to create the gMSA:

```
$DomainFqdn       = (Get-ADDomain).DNSRoot
$gMSAName         = "gMSA_CES"

$gMSAParameters = @{
    Name                                        = $gMSAName
    DNSHostName                                 = "$gMSAName.$DomainFqdn"
    PrincipalsAllowedToRetrieveManagedPassword = Get-ADGroup "GG-T0-CES Servers"
    Path                                        = "OU=Service Accounts,OU=Tier
0,OU=Admin,DC=corp,DC=michaelwaterman,DC=nl"
    Enabled                                     = $true
}

New-ADServiceAccount @gMSAParameters
```

Configure the Service Principal Names (SPNs)

To support Kerberos authentication, the HTTP Service Principal Names (SPNs) must be registered on the Group Managed Service Account (gMSA) used by the Certificate Enrollment Web Service. In addition, because CES forwards certificate requests to the Enterprise Certification Authority on behalf of authenticated users, the gMSA must be configured for Kerberos Constrained Delegation to the Certification Authority's HOST and RPCSS services.

The following PowerShell script automatically registers the required HTTP SPNs for all servers that are members of the GG-T0-CES Servers security group and configures the required delegation targets for the Enterprise Certification Authority.

This is a change to how I configured the CEP gMSA, in the case of the CES gMSA there could be what we know as protocol transition. This change is required when CES uses Username/Password or Client Certificate authentication. In these scenarios, CES must obtain a Kerberos service ticket on behalf of the authenticated user before forwarding the certificate request to the Enterprise Certification Authority. See a more detailed explanation after the code.

```
Import-Module ActiveDirectory

$GroupName = "GG-T0-CES Servers"
$gMSAName  = "gMSA_CES"
$CAName    = "lab-srv-02"
$EnableProtocolTransition = $true

$DomainFqdn = (Get-ADDomain).DNSRoot
$CAFqdn     = "$CAName.$DomainFqdn"

# Retrieve the CES gMSA as an AD object
$gMSA = Get-ADServiceAccount `
    -Identity $gMSAName `
    -Properties ServicePrincipalName, msDS-AllowedToDelegateTo

# Generate HTTP SPNs for all CES servers
[string[]]$SPNs = @(
    Get-ADGroupMember -Identity $GroupName |
        Where-Object ObjectClass -eq "computer" |
        ForEach-Object {

            $Computer = Get-ADComputer `
                -Identity $_.DistinguishedName `
                -Properties DNSHostName

            if ([string]::IsNullOrWhiteSpace($Computer.DNSHostName)) {
                Write-Warning "Computer '$($Computer.Name)' has no DNSHostName and
is skipped."
```

```
                return
            }

            "HTTP/$($Computer.Name)"
            "HTTP/$($Computer.DNSHostName)"
        }
) | Sort-Object -Unique

if ($SPNs.Count -eq 0) {
    throw "No valid HTTP SPNs were generated from group '$GroupName'."
}

# Configure HTTP SPNs on the CES gMSA
Set-ADServiceAccount `
    -Identity $gMSAName `
    -Replace @{
        ServicePrincipalName = [string[]]$SPNs
    }

# Configure Kerberos constrained delegation targets
[string[]]$AllowedToDelegateTo = @(
    "HOST/$CAName"
    "HOST/$CAFqdn"
    "RPCSS/$CAName"
    "RPCSS/$CAFqdn"
)

# Set-ADObject is more reliable for msDS-AllowedToDelegateTo
Set-ADObject `
    -Identity $gMSA.DistinguishedName `
    -Replace @{
        "msDS-AllowedToDelegateTo" = [string[]]$AllowedToDelegateTo
    }

# Enable Kerberos Protocol Transition
# This setting is equivalent to:
#   "Trust this user for delegation to specified services only"
#   "Use any authentication protocol"
#
# For a Kerberos-only CES endpoint, this can remain set to $false.
Set-ADAccountControl `
    -Identity $gMSA.DistinguishedName `
    -TrustedToAuthForDelegation $EnableProtocolTransition
```

Why does CES require kerberos constrained delegation?

Unlike the Certificate Enrollment Policy Web Service (CEP), the Certificate Enrollment Web Service (CES) performs operations *on behalf of* the authenticated user. While CEP only retrieves certificate enrollment policies from Active Directory using LDAP, CES must forward certificate requests to the Enterprise Certification Authority (CA) using Microsoft's native RPC/DCOM interfaces. This introduces what is commonly referred to as the *"second*

hop" problem. The Enterprise CA must evaluate the certificate request using the *identity of the authenticated user*, not the identity of the CES service account. To achieve this, CES uses *Kerberos Constrained Delegation* to securely delegate the user's identity to the Certification Authority.

When clients authenticate using Kerberos, the user's Kerberos ticket can be delegated directly to the CA. However, when using Username/Password or Client Certificate authentication, no Kerberos ticket is available. In these scenarios, CES must first perform Kerberos Protocol Transition before forwarding the request to the CA. This is why the "Use any authentication protocol" option (or the `TrustedToAuthForDelegation` setting) is required for these authentication methods. In summary:

Authentication Method	Constrained Delegation	Protocol Transition
Kerberos	Required	Not Required
Username/Password	Required	Required
Client Certificate	Required	Required

This architectural difference is one of the primary reasons Microsoft separated policy retrieval (CEP) from certificate enrollment (CES) into two distinct web services.

Install the Active Directory PowerShell module

Install the Active Directory PowerShell module on the future CES server. This module is required to install and validate the Group Managed Service Account.

```
Add-WindowsFeature RSAT--PowerShell
```

Install the Group Managed Service Account

Install the Group Managed Service Account on the CES server.

```
$gMSAName = "gMSA_CES"

try {
    Install-ADServiceAccount `
        -Identity $gMSAName `
        -ErrorAction Stop
    Write-Host "Successfully installed gMSA '$gMSAName'." -ForegroundColor Green
}
catch {
    if ($_.Exception.Message -match "already exists|already installed") {
        Write-Host "gMSA '$gMSAName' is already installed." -ForegroundColor Yellow
    }
```

```
    else {
        throw
    }
}
```

Validate the Group Managed Service Account

Before continuing with the installation, verify that the Group Managed Service Account was successfully installed and can retrieve its managed password.

```
if (Test-ADServiceAccount -Identity $gMSAName) {
    Write-Host "gMSA validation successful." -ForegroundColor Green
}
else {
    throw "gMSA validation failed."
}
```

Add the gMSA to the IIS_IUSRS group

Finally, add the Group Managed Service Account to the local *IIS_IUSRS* group. This grants IIS permission to use the account for the application pool.

```
$Parameters = @{
    Group  = "IIS_IUSRS"
    Member = "$((Get-ADDomain).NetBIOSName)\gMSA_CES$"
}

Add-LocalGroupMember @Parameters
```

Installing Internet Information Services

The Certificate Enrollment Policy Web Service is hosted in Internet Information Services (IIS). Although the CES installation wizard automatically installs the required IIS components, I prefer installing IIS separately. This provides access to the IIS management tools and makes it easier to verify and customize the installation if needed. To install IIS together with the management tools, run:

```
$Features = @{
    Name                   = "Web-Server"
    IncludeManagementTools = $true
}

Install-WindowsFeature @Features
```

Alternatively, if you prefer to install only the required IIS components, use the following command:

```
$Features = @(
  "Web-Server"
        "Web-WebServer"
        "Web-Common-Http"
        "Web-Default-Doc"
        "Web-Dir-Browsing"
        "Web-Http-Errors"
        "Web-Static-Content"
        "Web-Health"
        "Web-Http-Logging"
        "Web-Performance"
        "Web-Stat-Compression"
        "Web-Security"
        "Web-Filtering"
        "Web-Mgmt-Tools"
        "Web-Mgmt-Console"
)

Install-WindowsFeature `
    -Name $Features
```

Configure the Windows firewall

The Certificate Enrollment Web Service communicates exclusively over HTTP and HTTPS. Enable the required Windows Firewall rules before continuing with the installation.

```
Enable-NetFirewallRule -Name "IIS-WebServerRole-HTTP-In-TCP"
Enable-NetFirewallRule -Name "IIS-WebServerRole-HTTPS-In-TCP"
```

With Active Directory prepared and IIS installed, I can now deploy the *Certificate Enrollment Web Service (CES)*. Installing the Windows feature adds the required CES components to the server, after which I'll configure the supported authentication methods and connect the service to the Enterprise Certification Authority. Install the Certificate Enrollment Web Service by running the following PowerShell command:

```
$Features = @{
    Name                   = "ADCS-Enroll-Web-Svc"
    IncludeManagementTools = $false
}

Install-WindowsFeature @Features
```

Configure Kerberos authentication

The first authentication method I'll configure is Kerberos, which is the preferred option for domain-joined clients. During the installation, the script automatically locates a valid TLS certificate matching the server's fully qualified domain name (FQDN) and uses its thumbprint to secure the HTTPS endpoint.

Unlike the Certificate Enrollment Policy Web Service (CEP), the Certificate Enrollment Web Service (CES) must also be configured to communicate with the Enterprise Certification Authority. During the installation, the target CA and the required application pool identity are therefore specified as part of the deployment.

```
$CesFqdn = "$env:COMPUTERNAME.$((Get-ComputerInfo).CSDomain)"

$CAServerFqdn = "lab-srv-02.corp.michaelwaterman.nl"
$CACommonName = "Corp-Enterprise-CA" # Replace with the actual CA name
$CAConfig     = "$CAServerFqdn\$CACommonName"

$Cert = Get-ChildItem -Path "Cert:\LocalMachine\My" |
    Where-Object {
        $_.DnsNameList.Unicode -contains $CesFqdn -and
        $_.HasPrivateKey -and
        $_.NotBefore -le (Get-Date) -and
        $_.NotAfter  -gt (Get-Date) -and
        $_.EnhancedKeyUsageList.ObjectId -contains "1.3.6.1.5.5.7.3.1"
    } |
    Sort-Object NotAfter -Descending |
    Select-Object -First 1

if (-not $Cert) {
    throw "No valid TLS certificate for '$CesFqdn' was found in
Cert:\LocalMachine\My."
}

Write-Host "Using TLS certificate:" -ForegroundColor Cyan
$Cert | Format-List Subject, Thumbprint, NotAfter, HasPrivateKey

$CesConfigurationParameters = @{
    CAConfig               = $CAConfig
    AuthenticationType     = "Kerberos"
    SSLCertThumbprint      = $Cert.Thumbprint
    ApplicationPoolIdentity = $true
    Force                  = $true
}

Install-AdcsEnrollmentWebService @CesConfigurationParameters
```

Configure Username/Password authentication

Username/Password authentication enables non-domain-joined systems, such as workgroup or DMZ servers, to retrieve enrollment policies over HTTPS. The installation process is identical to Kerberos authentication, with only the authentication method changing. `AuthenticationType = "UserName"`.

```
$CesConfigurationParameters = @{
    CAConfig               = $CAConfig
```

```
    AuthenticationType      = "Kerberos"
    SSLCertThumbprint       = $Cert.Thumbprint
    Force                   = $true
}
```

Note! The ApplicationPoolIdentity is absent here as it can only be set once.

Note! Username/Password authentication requires Kerberos Protocol Transition. Enable the `TrustedToAuthForDelegation` flag on the CES gMSA before installing the service.

Configure client certificate authentication

Client Certificate authentication allows clients to authenticate using an existing certificate instead of Kerberos or user credentials. This method is commonly used for certificate renewal scenarios and environments where passwordless authentication is preferred. The installation process is identical to Kerberos or Username / Password authentication, with only the authentication method changing. `AuthenticationType = "Certificate"`.

```
$CepConfigurationParameters = @{
    AuthenticationType = "Certificate"
    SSLCertThumbprint   = $Cert.Thumbprint
    Force               = $true
}
```

Note! The ApplicationPoolIdentity is absent here as it can only be set once.

Note! Certificate authentication requires Kerberos Protocol Transition. Enable the `TrustedToAuthForDelegation` flag on the CES gMSA before installing the service.

Key-Based renewal

The Certificate Enrollment Web Service optionally supports Key-Based Renewal, allowing clients to renew an existing certificate using possession of the current private key as proof of identity. This can be enabled using the `-AllowKeyBasedRenewal` parameter during installation. It is primarily intended for certificate renewal scenarios and is not required for initial certificate enrollment.

Note! If you plan to support key-based certificate renewal, enable Key-Based Renewal on both the Certificate Enrollment Policy Web Service (CEP) and the Certificate Enrollment Web Service (CES). This ensures that the enrollment policy presented to clients matches the capabilities of the enrollment service.

Renewal only

By default, the Certificate Enrollment Web Service (CES) accepts both new certificate enrollment and certificate renewal requests. In some environments, however, it may be

desirable to expose a dedicated CES endpoint that only accepts renewal requests. This behavior can be enabled by specifying the `-RenewalOnly` parameter during installation.

Configure the IIS Application Pool Identity

By default, the WSEnrollmentServer application pool runs under the built-in ApplicationPoolIdentity account. To improve security and eliminate manual password management, I'll configure the application pool to use the dedicated Group Managed Service Account (gMSA) created earlier for the Certificate Enrollment Web Service (CES). Run the following PowerShell command to update the application pool identity:

```
$ApplicationPool = "WSEnrollmentServer"
$gMSAName        = "gMSA_CES"
$DomainNetBIOS   = (Get-ADDomain).NetBIOSName

Import-Module WebAdministration

Set-ItemProperty `
    -Path "IIS:\AppPools\$ApplicationPool" `
    -Name processModel.identityType `
    -Value 3

Set-ItemProperty `
    -Path "IIS:\AppPools\$ApplicationPool" `
    -Name processModel.userName `
    -Value "$DomainNetBIOS\$gMSAName$"

Set-ItemProperty `
    -Path "IIS:\AppPools\$ApplicationPool" `
    -Name processModel.password `
    -Value ""

Restart-WebAppPool -Name $ApplicationPool
```

Note! All the binaries are installed in the directory:"C:\Windows\systemdata\CES".

Validating the certificate enrollment web service

With the installation complete, the final step is to verify that each Certificate Enrollment Web Service (CES) endpoint is functioning correctly. In this section, I'll validate the Kerberos, Username/Password, and Client Certificate endpoints using `certutil`. A successful response confirms that CES is correctly configured and able to communicate with the Enterprise Certification Authority.

Validate kerberos authentication

Use the following command from a domain-joined computer to verify that the Kerberos-enabled CES endpoint is accessible.

```
$CepFqdn = "lab-srv-04.corp.michaelwaterman.nl"
$CesUrl  = "https://$CepFqdn/Corp-Enterprise-CA_CES_Kerberos/service.svc/CES"

$Output = & certutil.exe `
    -ping `
    -kerberos `
    -config $CesUrl `
    CES

if ($LASTEXITCODE -eq 0) {
    Write-Host "CES '$CesUrl' is responding successfully." -ForegroundColor Green
}
else {
    throw "CES test failed:`n$($Output -join "`n")"
}
```

The response should be: “`CES 'https://lab-srv-04.corp.michaelwaterman.nl/Corp-Enterprise-CA_CES_Kerberos/service.svc/CES' is responding successfully.`“

Validate Username/Password Authentication

The following command validates the Username/Password CES endpoint using explicit Active Directory credentials.

```
$UserName = "CORP\Superuser"
$Password = "P@ssw0rd!"
$CesUrl   = "https://lab-srv-04.corp.michaelwaterman.nl/Corp-Enterprise-
CA_CES_UsernamePassword/service.svc/CES"

$Output = & certutil.exe `
    -UserName $UserName `
    -p $Password `
    -ping `
    -config $CesUrl `
    CES

if ($LASTEXITCODE -eq 0) {

    Write-Host "Successfully connected to the CES endpoint using username/password
authentication." -ForegroundColor Green

}
else {

    throw "Failed to connect to the CES endpoint.`n$($Output -join "`n")"
```

```
}
```

Note! *This test will trigger a Windows Defender warning because credentials are supplied on the command line. This behavior is expected in this lab scenario*.

Validate Client Certificate Authentication

Finally, validate the Client Certificate authentication endpoint using a valid client authentication certificate. That is if you have a client authentication certificate available.

```
$ClientCertificateThumbprint = "<Thumbprint>"
$CesFqdn                     = "lab-srv-04.corp.michaelwaterman.nl"
$CesUrl                      = "https://$CesFqdn/Corp-Enterprise-
CA_CES_Certificate/service.svc/CES"

$ClientCertificate = Get-Item `
    -Path "Cert:\CurrentUser\My\$ClientCertificateThumbprint" `
    -ErrorAction Stop

$Output = & certutil.exe `
    -ClientCertificate $ClientCertificate.Thumbprint `
    -ping `
    -config $CesUrl `
    CES 2>&1

if ($LASTEXITCODE -eq 0) {

    Write-Host "Certificate authentication to CES succeeded." -ForegroundColor
Green

}
else {

    throw "Certificate authentication to CES failed.`n$($Output -join "`n")"

}
```

Optional Validation

Although `certutil -ping` confirms that the service is reachable, it does not perform an actual certificate enrollment. In the next article, I'll configure both domain-joined and workgroup clients and perform end-to-end certificate enrollment using CEP and CES.

Summary

Whoop whoop! You have successfully deployed the Certificate Enrollment Web Service (CES) and completed the Microsoft Certificate Enrollment Services architecture. Together with the Certificate Enrollment Policy Web Service (CEP) from the previous article, your

environment is now capable of securely processing certificate enrollment requests over HTTPS using the XCEP and WSTEP protocols.

In this article, I prepared Active Directory, configured a dedicated Group Managed Service Account (gMSA), installed IIS, deployed the Certificate Enrollment Web Service, configured support for Kerberos, Username/Password, and Client Certificate authentication, and validated each endpoint using `certutil`. We also explored why CES requires Kerberos Constrained Delegation and, depending on the selected authentication method, Kerberos Protocol Transition.

While the infrastructure is now fully operational, I have not yet configured any clients to use it. In the next article, I'll bring everything together by configuring both domain-joined and workgroup clients, registering Certificate Enrollment Policy Servers using Group Policy and PowerShell, and performing end-to-end certificate enrollment using the Microsoft Certificate Enrollment Services infrastructure.